# Security Audit Report - OWASP Juice Shop

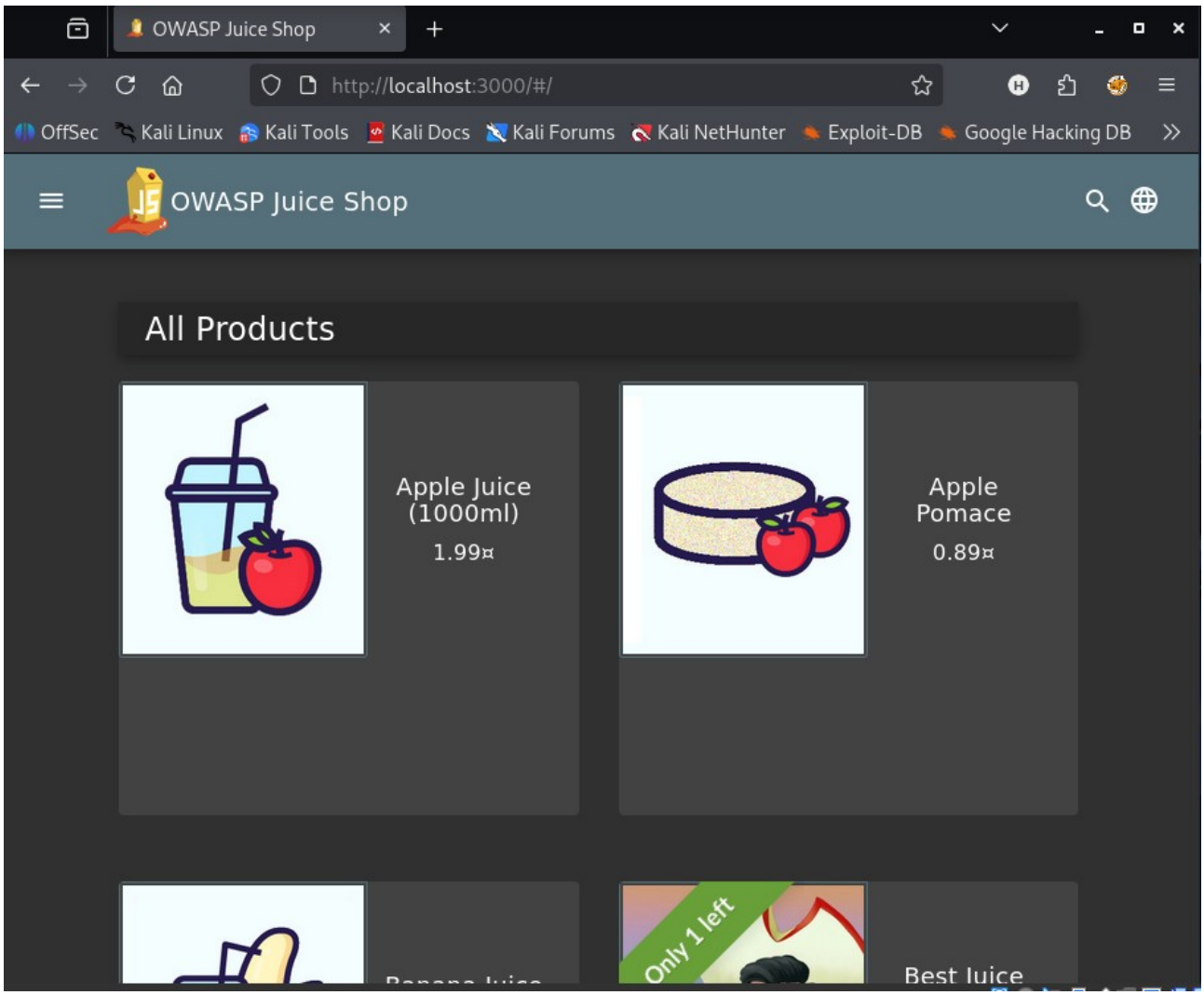
**Overview:**
This project presents a security audit performed on the deliberately vulnerable web application OWASP Juice Shop. The objective of this assessment was to identify common web vulnerabilities using automated penetration testing tools. The vulnerability scanning and analysis were performed using OWASP ZAP.

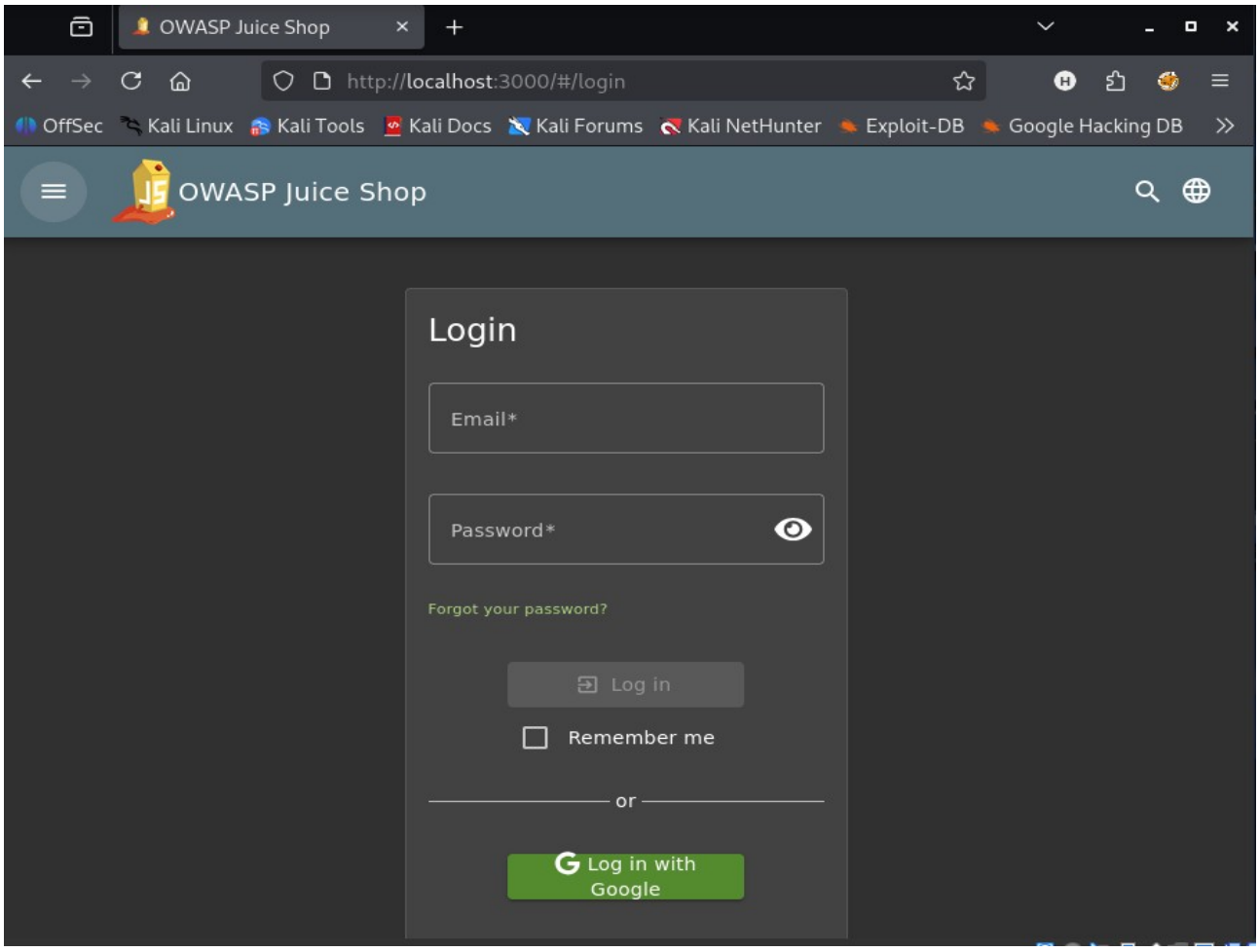
**Objectives:**
- Identify vulnerabilities in a web application
- Perform automated security scanning
- Analyze security weaknesses such as SQL Injection and misconfigurations
- Provide mitigation recommendations

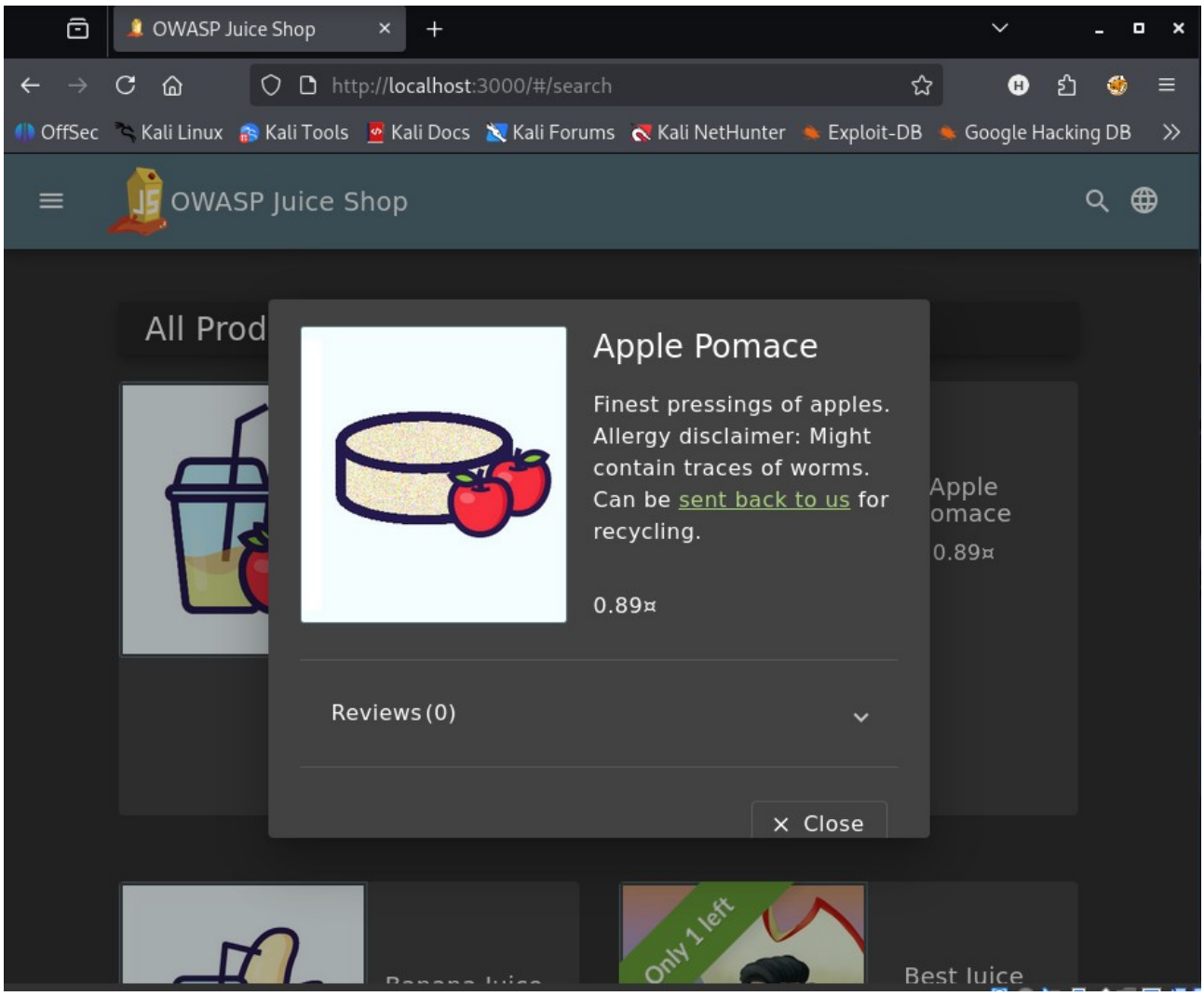
**Tools Used:**
- Kali Linux
- OWASP ZAP
- OWASP Juice Shop
- Firefox Browser

**Testing Methodology**
1. Environment Setup
The vulnerable application was deployed locally using a containerized environment and accessed through the browser.

**Target URL:**
http://localhost:3000


OWASP Juice Shop
http://localhost:3000/#/
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
OWASP Juice Shop
All Products
Apple Juice (1000ml)
1.99¤
Apple Pomace
0.89¤
Only 1 left
Best Juice


OWASP Juice Shop
http://localhost:3000/#/login
OffSec
Kali Linux
Kali Tools
Kali Docs
Kali Forums
Kali NetHunter
Exploit-DB
Google Hacking DB
OWASP Juice Shop
Login
Email*
Password*
Forgot your password?
Log in
Remember me
or
Log in with Google

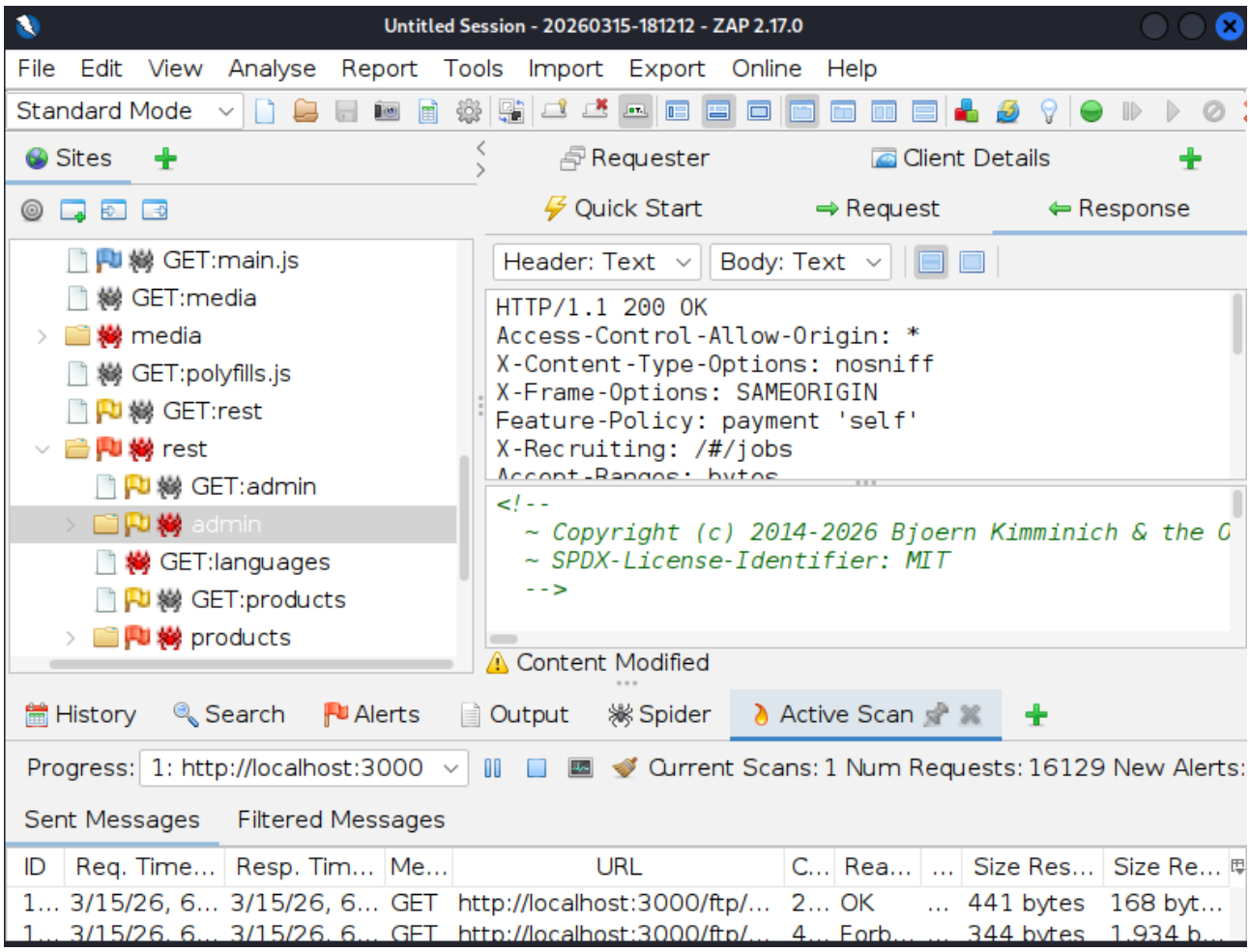



**2. Proxy Configuration:**
The browser proxy was configured to route traffic through OWASP ZAP.

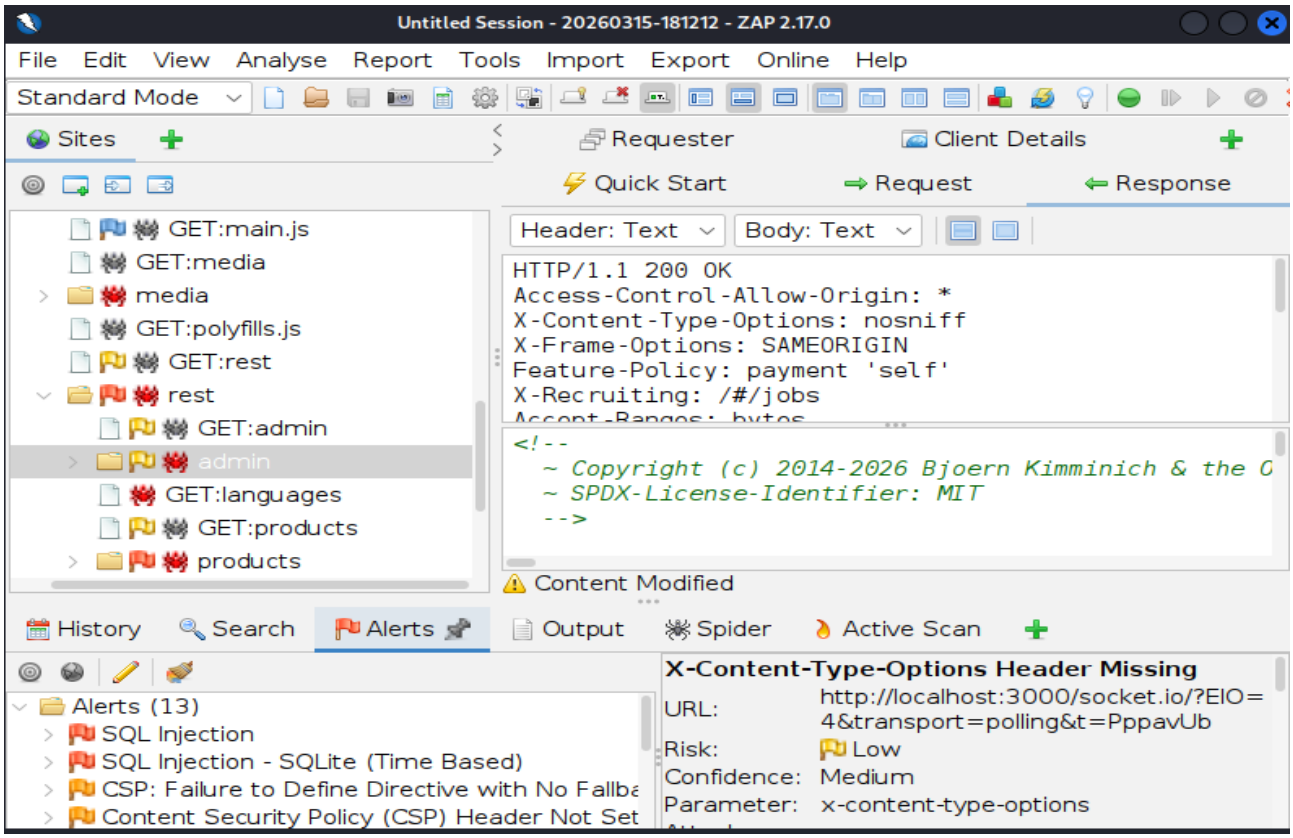
Proxy IP: 127.0.0.1
Port: 8080
This allowed the tool to intercept and analyze HTTP requests and responses.

**3. Automated Vulnerability Scan**

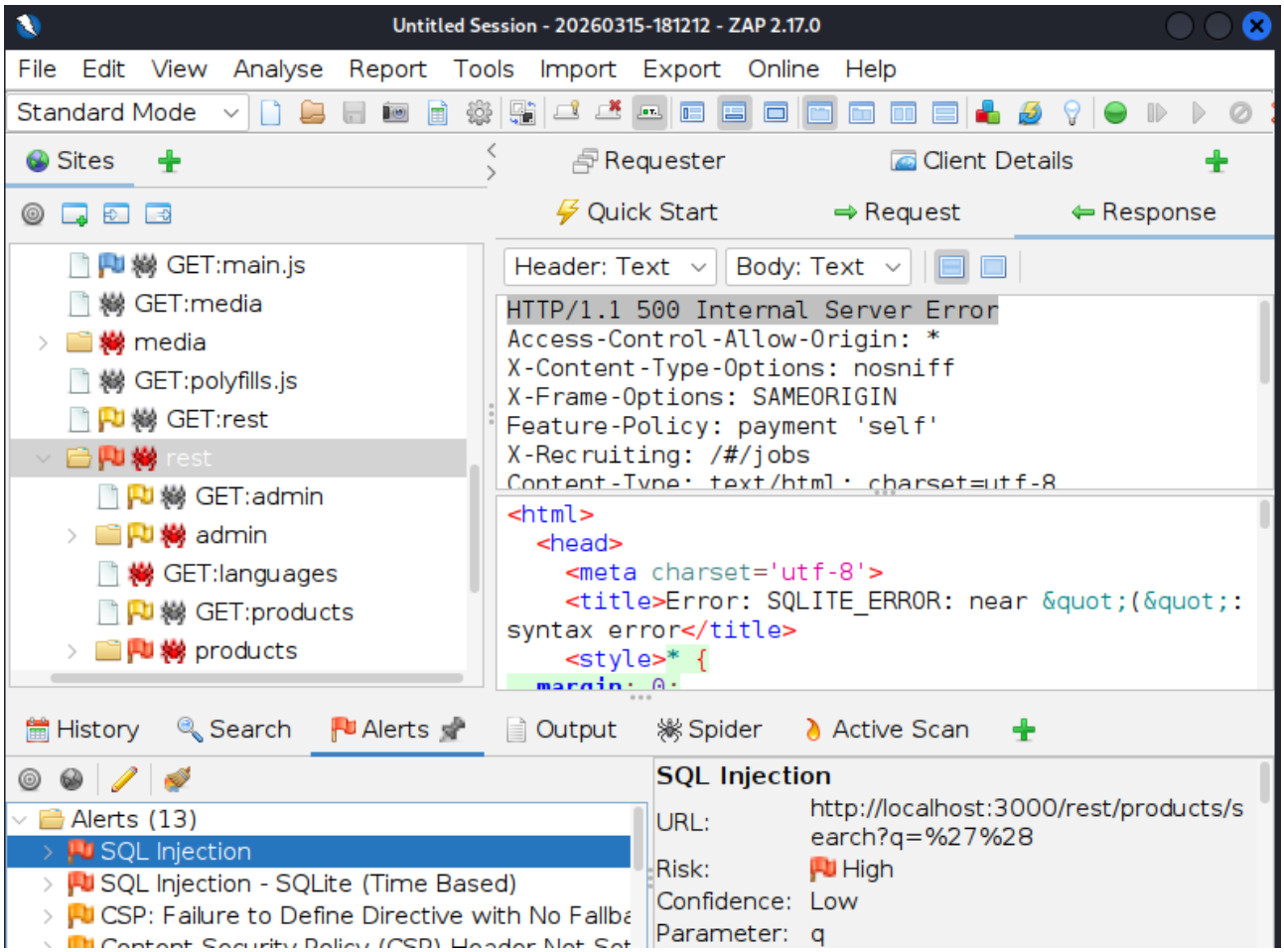
OWASP ZAP was used to perform an automated spider and active scan of the application.
The scanner explored the application structure and tested input fields for common vulnerabilities.




## Vulnerability Findings



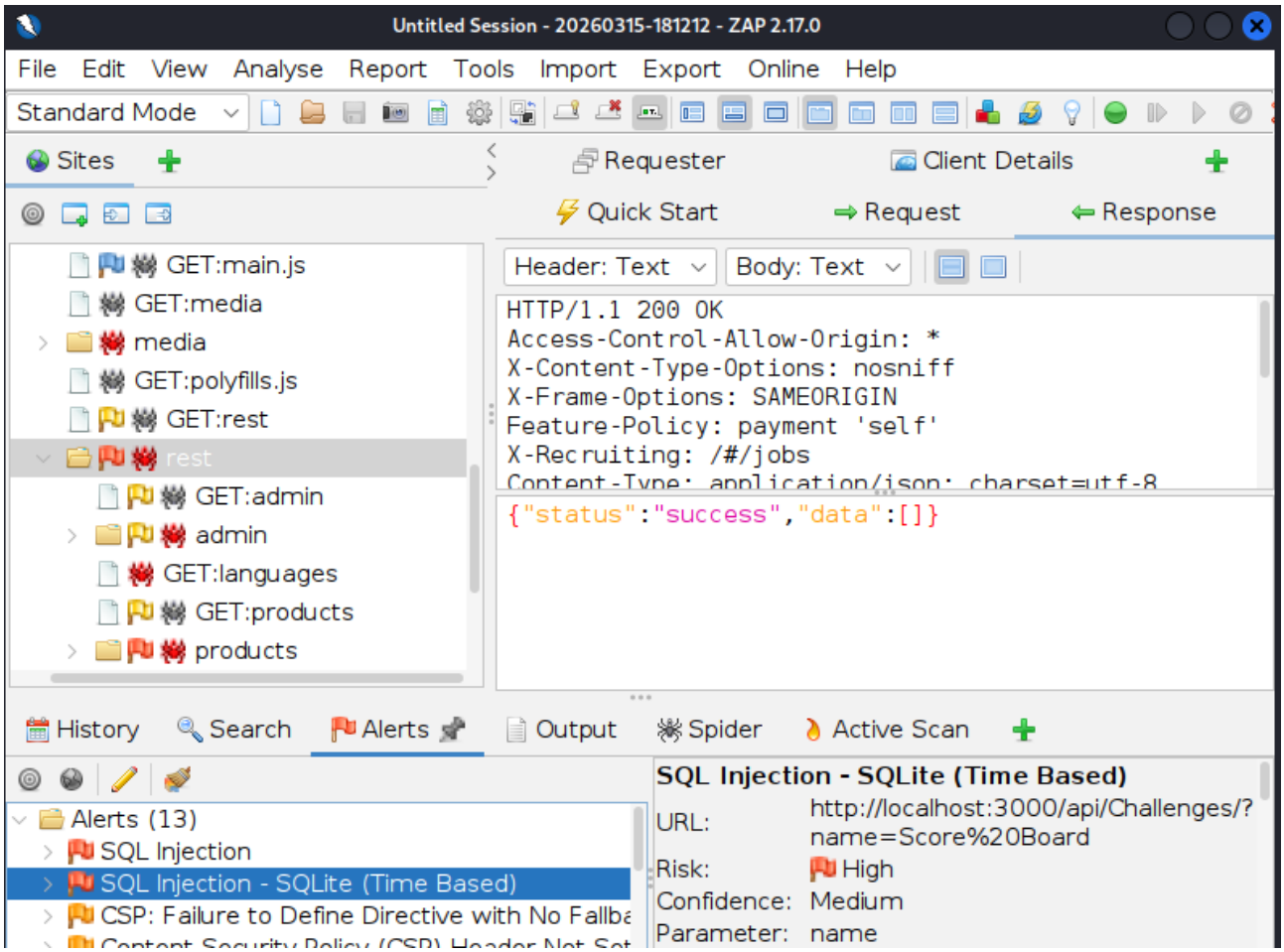
**1. SQL Injection:**
- Risk Level: High
- SQL Injection occurs when user input is improperly validated and is directly included in database queries.
- This vulnerability may allow attackers to manipulate database queries and potentially extract or modify sensitive data.




**Impact:**
- Database data exposure
- Authentication bypass
- Data manipulation

**Mitigation:**
- Use prepared statements
- Implement input validation
- Use parameterized database queries

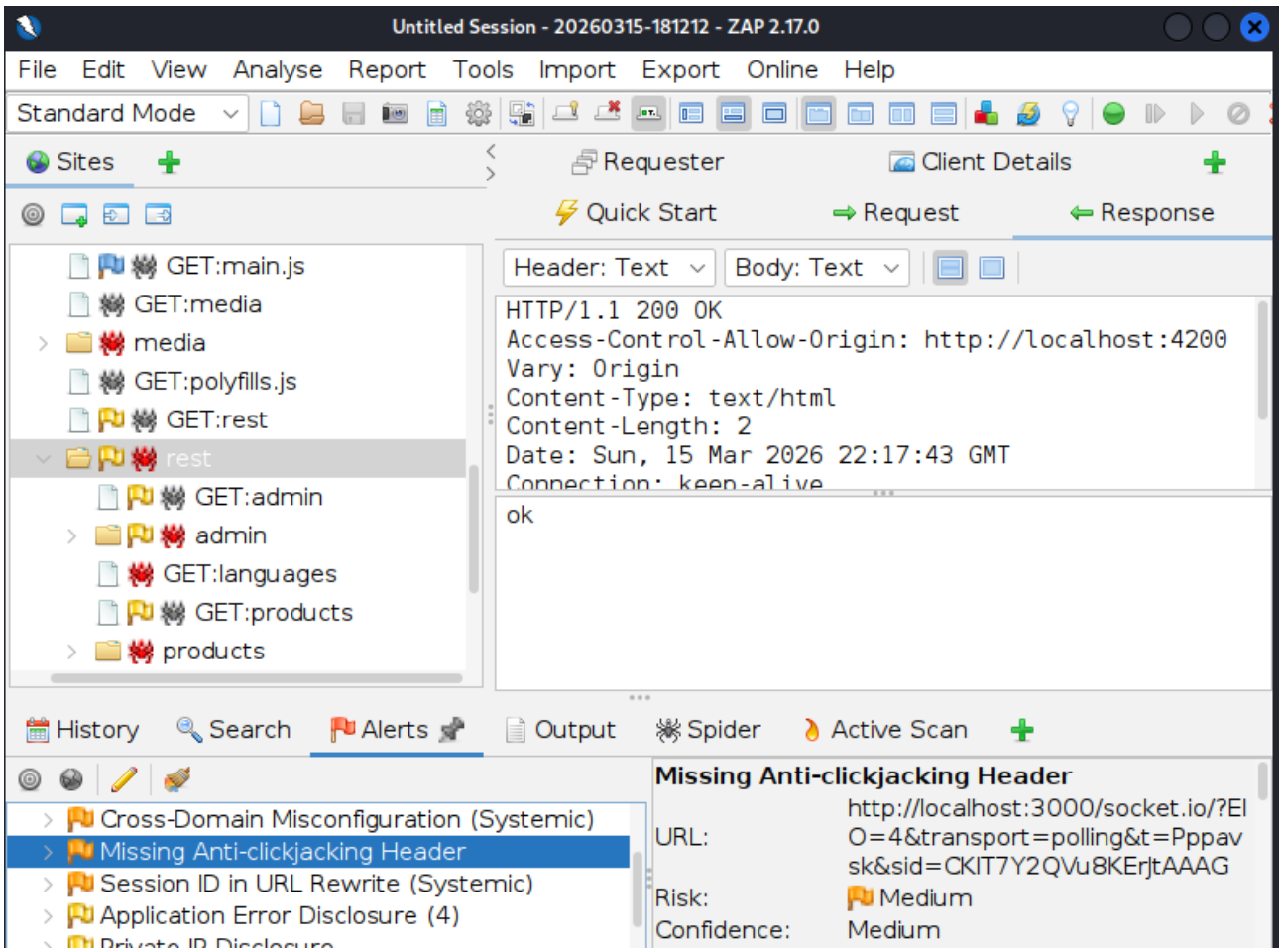
**2. SQL Injection – SQLite Time Based:**
- Risk Level: High
- Time-based SQL Injection is a blind injection technique where attackers infer database responses based on delayed server responses.




**Impact:**
- Database information disclosure
- exploitation without visible errors

**Mitigation:**
- parameterized queries
- strict input validation

3. **Missing Anti-Clickjacking Header:**
- Risk Level: Medium
- The application does not include the X-Frame-Options header, making it vulnerable to clickjacking attacks.

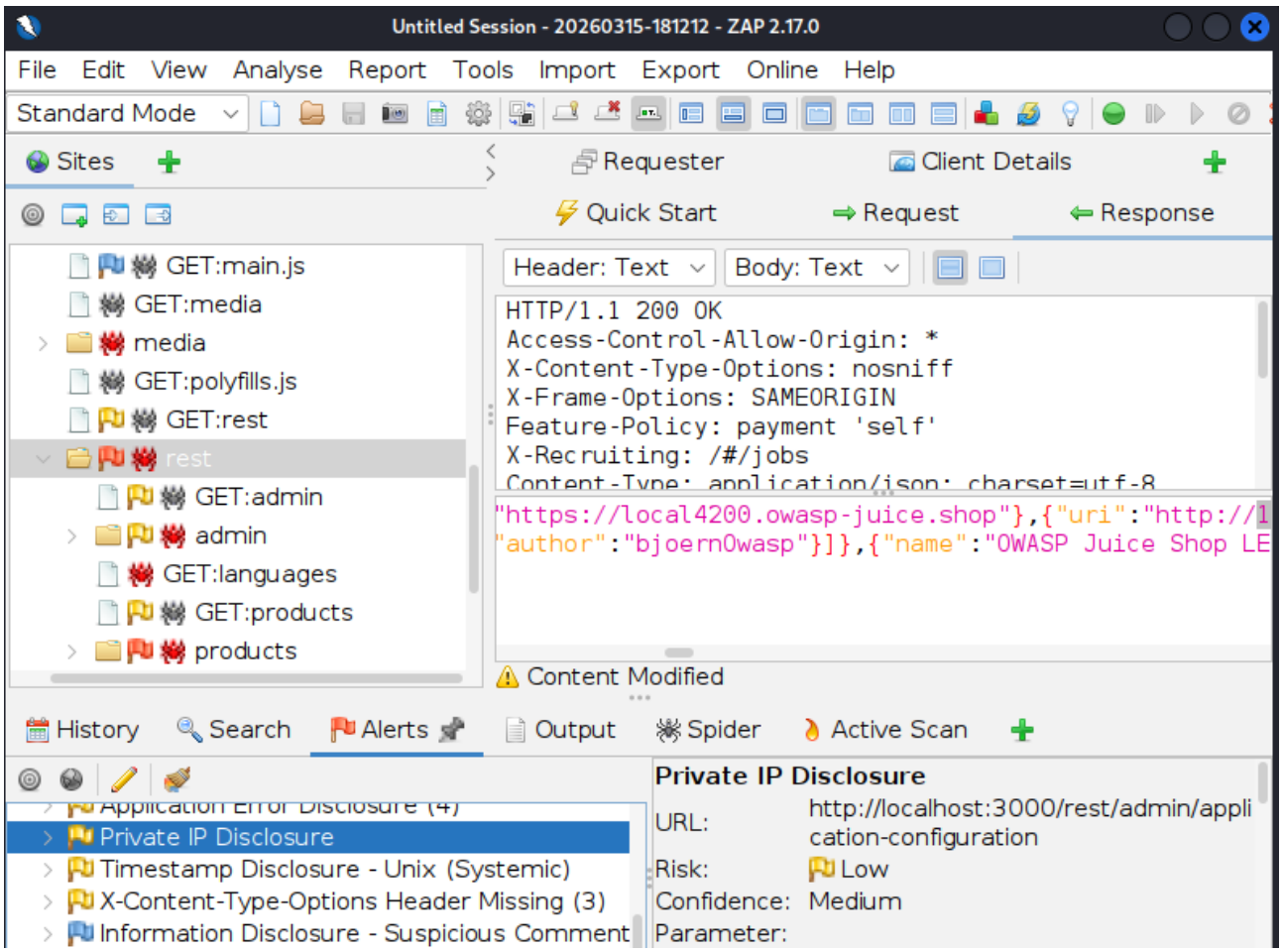



**Impact**

- Attackers may trick users into clicking hidden elements embedded inside malicious websites.
- Mitigation
- X-Frame-Options: DENY

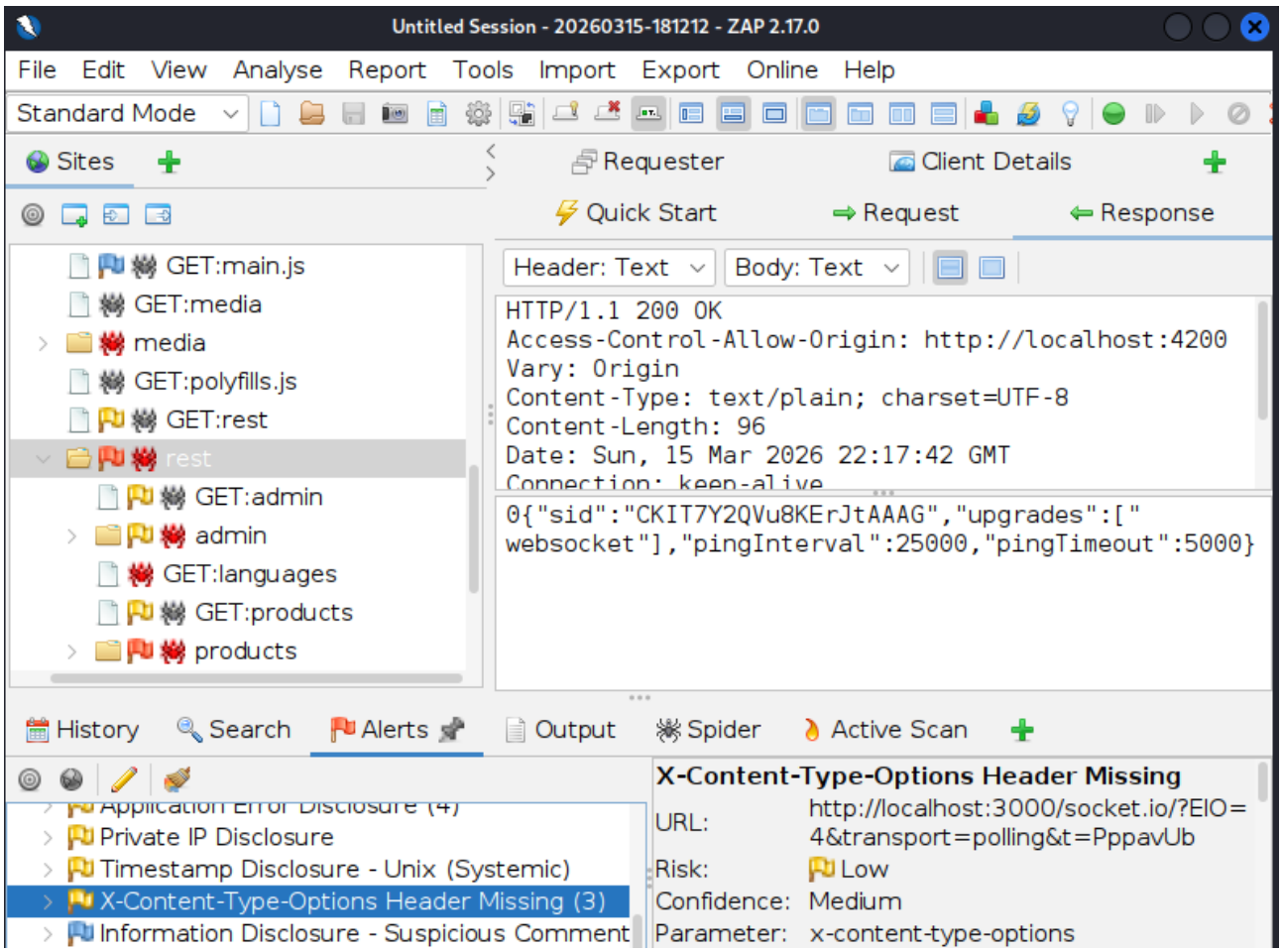
**4. Private IP Disclosure:**

- Risk Level: Low
- The application reveals internal IP addresses in server responses.




**Impact:**
- This information could assist attackers in mapping internal infrastructure.
- Mitigation
- Avoid exposing internal network details in responses.

**5. X-Content-Type Header Missing:**
- Risk Level: Low- The application does not define the X-Content-Type-Options header.




**Impact:**
- Browsers may perform MIME type sniffing which can lead to security risks.
- Mitigation
- X-Content-Type-Options: nosniff

**Recommendations:**
- Implement input validation and output encoding
- Use prepared database queries
- Configure proper HTTP security headers
- Perform regular vulnerability scanning
- Conduct secure code reviews

**Conclusion:**
The penetration testing assessment revealed multiple security vulnerabilities within the application. These findings demonstrate the importance of secure coding practices and proper security configuration to protect web applications from potential attacks.